Taller: Tablas Hash

Objetivo:
Implementar una tabla hash con direccionamiento abierto respetando su invariante, usar una tabla hash para bajar la complejidad de un problema de conteo, y usar el factor de carga y el análisis amortizado para decidir los parámetros de una tabla. En cada problema se pide describir la secuencia de sondeo o la idea, el invariante, los casos borde y la complejidad antes de implementarlo en Python leyendo la entrada estándar.
Parte 1 (Problemas 1 y 2): implementación. Parte 2: análisis.
Los casos de prueba están en la carpeta pruebas/ (casilleros.in y casilleros.out; pares.in y pares.out; pares_grande.in y pares_grande.out). Su programa debe producir exactamente la salida del archivo .out cuando lee el archivo .in por la entrada estándar.

Problema 1: Casilleros del Gimnasio

Contexto y propósito:

El gimnasio de la universidad tiene m casilleros, numerados de 0 a m − 1, y asigna uno a cada estudiante con un sistema automático: a partir del código del estudiante, el sistema calcula qué casilleros revisar y en qué orden. Cuando un estudiante deja de usar su casillero, el sistema no lo marca como «nunca usado», sino como «liberado». Lo que se pretende es que adapte el direccionamiento abierto de la guía (ejercicio 2) a otra secuencia de sondeo, el doble hashing; que identifique qué invariante permite detener una búsqueda en un casillero vacío y por qué ese invariante obliga a distinguir «liberado» de «vacío»; y que descubra por qué asignar un casillero exige terminar la búsqueda antes de usar uno liberado. La salida pide el casillero y el número de casilleros revisados en cada operación: solo la produce un programa que siga exactamente las reglas, así que un dict de Python no sirve para resolverlo.

Enunciado:
Cada casillero está VACÍO (nunca se ha usado), OCUPADO por el código de un estudiante, o LIBERADO (estuvo ocupado y ya no lo está). Para el código k, el sistema revisa los casilleros $p(i) = (h1(k) + i \cdot h2(k)) \bmod m$, para $i = 0, 1, \ldots, m - 1$, donde $h1(k) = k \bmod m$ y $h2(k) = 1 + (k \bmod (m - 1))$. Como m es primo, esa secuencia pasa por los m casilleros. Las operaciones son:
- B k (buscar): revisa la secuencia hasta encontrar un casillero OCUPADO por k, encontrar un casillero VACÍO o haber revisado los m casilleros. Si encontró k en el casillero p, imprime p y el número s de casilleros revisados; si no, imprime NO y s.
- I k (asignar): hace la misma búsqueda. Si k ya tiene el casillero p, imprime YA, p y s. Si no lo tiene, le asigna el primer casillero LIBERADO que haya visto durante la búsqueda o, si no vio

ninguno, el casillero VACÍO en el que se detuvo, e imprime ese casillero y s. Si revisó los m casilleros sin ver ninguno LIBERADO ni VACÍO, imprime LLENO y s.
- E k (liberar): hace la misma búsqueda. Si encontró k en el casillero p, lo marca LIBERADO e imprime p y s; si no, imprime NO y s.
En todas las operaciones, s cuenta los casilleros revisados, incluido aquel en el que se detuvo la búsqueda. Al empezar cada caso, todos los casilleros están VACÍOS.

Entrada:
- Un entero T ($1 \le T \le 10$), el número de casos de prueba
- Para cada caso: una línea con m y q (m primo, $3 \le m \le 211$; $1 \le q \le 1000$), seguida de q líneas, cada una con una operación (B, I o E) y un código k ($0 \le k \le 10^9$)

Salida:
- Para cada caso, una línea «Caso c:» (c empieza en 1) y luego una línea por operación, con el formato descrito

Ejemplo:
Entrada:
1
7 8
I 10
I 17
I 24
B 24
E 17
B 24
I 59
B 59
Salida:
Caso 1:
3 1
2 2
4 2
4 2
2 2
4 2
2 3
2 2

En la séptima operación, el código 59 revisa el casillero 3 (ocupado por 10), el 2 (liberado:

se anota) y el 1 (vacío: 59 no tenía casillero); se le asigna el 2, después de revisar 3 casilleros.

1. Escriba la secuencia de casilleros que revisa el código 59 con m = 7 y explique por qué, si m es primo, la secuencia p(0), …, p(m − 1) pasa por los m casilleros. ¿Qué podría pasar con m = 8 y un código cuyo h2 sea par?

2. Escriba el invariante de la tabla que permite detener una búsqueda en un casillero VACÍO (guía, ejercicio 2, pregunta 2). Muestre, con una secuencia de operaciones concreta, qué respuesta incorrecta daría el sistema si liberar dejara el casillero VACÍO.

3. Explique por qué I k no puede asignar el primer casillero LIBERADO apenas lo encuentra. Construya una secuencia de no más de cinco operaciones, con m = 7, en la que esa versión le deje a un mismo código dos casilleros.

4. Calcule el número máximo de casilleros que revisa una operación y la complejidad del programa para un caso con m casilleros y q operaciones. ¿Por qué los casilleros LIBERADOS hacen más lentas las búsquedas aunque haya pocos estudiantes inscritos? ¿Qué haría para evitarlo?

5. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada) y pruébela con pruebas/casilleros.in: su salida debe coincidir con pruebas/casilleros.out.

Problema 2: Pagos que Suman K

Contexto y propósito:

Al conciliar las cuentas del mes, el área contable de una empresa tiene la lista de los N pagos recibidos y busca parejas de pagos que sumen exactamente un valor K: por ejemplo, el valor de una factura que un cliente pudo haber dividido en dos transferencias. Es la versión de conteo del ejercicio 4 de la guía, que a su vez es el ejercicio 4 del Parcial 1. Lo que se pretende es que pase de «¿existe un par?» a «¿cuántos pares hay?»; que descubra qué debe guardar la tabla hash para contar bien cuando hay valores repetidos (un conjunto ya no basta); que trate el caso en que un pago es exactamente la mitad de K; y que compare, en tiempo y memoria, la solución con tabla hash contra la de ordenar y usar dos punteros. Con N = 200 000, probar todos los pares significa revisar unos $2 \cdot 10^{10}$ pares.

Enunciado:
Dada una lista de N enteros a(1), …, a(N) y un entero K, cuente cuántos pares de índices (i, j), con i < j, cumplen a(i) + a(j) = K. Dos pares son distintos si difieren en algún índice, aunque los valores sean iguales.

Entrada:
- Un entero T ($1 \le T \le 20$), el número de casos de prueba
- Para cada caso: una línea con N y K ($1 \le N \le 200\,000$, $|K| \le 2 \cdot 10^9$) y una línea con los N enteros ($|a(i)| \le 10^9$). La suma de los N de todos los casos no pasa de 200 000.

Salida:
- Para cada caso, una línea con el número de pares. La respuesta puede ser mayor que 2^{31}.

Ejemplo:
Entrada:
2
6 10
8 3 5 9 1 7
5 6
3 3 3 3 3
Salida:
2
10

En el primer caso los pares son 3 + 7 y 9 + 1; el 5 no forma par consigo mismo. En el

segundo, cualquier par de los cinco 3 suma 6, y hay 10 pares así.

1. Escriba la solución de fuerza bruta y su complejidad. Estime cuántos pares revisa con N = 200 000 y cuánto tardaría en Python si revisa unos 10^7 pares por segundo.

2. Describa la solución de una pasada con tabla hash: ¿qué guarda la tabla y por qué un set no alcanza?, ¿qué invariante cumple la tabla al empezar a revisar a(j)?, ¿en qué orden se consulta y se actualiza la tabla en cada paso, y por qué ese orden?

3. Explique cómo trata su solución el caso a(j) = K − a(j). Verifique a mano, paso a paso, que con el segundo caso del ejemplo obtiene 10.

4. Calcule la complejidad en tiempo y en memoria, e indique bajo qué suposición vale. Compárela con ordenar y usar dos punteros: ¿cuál usa menos memoria?, ¿cómo tiene que contar esa otra solución los pares cuando hay valores repetidos?

5. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada) y pruébela con pruebas/pares.in. Luego ejecútela con pruebas/pares_grande.in (N = 40 000) y compare con lo que tarda la fuerza bruta sobre el mismo archivo.

Parte 2: Análisis

Contexto y propósito:
Lo que se pretende es que use las fórmulas de la guía para decidir los parámetros de una tabla hash, no solo para calcular: qué factor de carga tolerar, cómo crecer y qué m escoger. Las preguntas conectan con los dos problemas de la Parte 1.
Instrucciones:
Complete la tabla con el número esperado de sondeos (guía, secciones 5 y 6). Para el doble hashing del problema 1 use la aproximación del sondeo uniforme: $1/(1 - \alpha)$ sondeos en una búsqueda sin éxito.

α	Encadenamiento 1 + α	Sondeo lineal, con éxito	Sondeo lineal, sin éxito	Doble hashing, sin éxito
0.25				
0.50				
0.75				
0.90				
0.99				

1. Con la tabla completa, ¿qué factor de carga máximo permitiría en cada esquema si una búsqueda sin éxito no debe revisar, en promedio, más de 5 celdas? Relacione su respuesta con los umbrales que usan Python (2/3) y Java (0.75).

2. Una tabla empieza con m = 1 y duplica m cada vez que está llena (α = 1). Cuente el costo total de insertar $N = 2^k$ claves, contando 1 por inserción y 1 por cada clave reubicada, y muestre que es menor que 3N. Repita suponiendo que, en lugar de duplicar, suma 100 casillas: ¿de qué orden es ahora el costo total?

3. Las claves de un sistema son múltiplos de 6. Para m = 12, m = 13 y m = 64, ¿cuántos cajones distintos usa h(k) = k mod m? Justifique con el máximo común divisor. ¿Cambia su respuesta si las claves son múltiplos de 6 más 1?

4. En el problema 2 las claves de la tabla son enteros, cuyo hash en Python es fijo (hash(5) == 5). Explique por qué Python decidió hacer aleatorio el hash de las cadenas y no el de los enteros, y qué riesgo correría un servidor web que guardara en un dict los nombres de los campos de un formulario si el hash de las cadenas fuera fijo (guía, sección 9).